

Technical Report

AI-Assisted DBC Generation and CAN Data Visualization Using SocketCAN

1. Signal Design

A simple Vehicle Information Network was designed using five required automotive signals and one additional signal for Challenge 3.

CAN ID	Signal	Start Bit	Length	Scale	Offset	Unit
0x100	Vehicle Speed	0	16	0.01	0	km/h
0x100	Engine RPM	16	16	1	0	rpm
0x101	Coolant Temperature	0	8	1	-40	°C
0x101	Fuel Level	8	8	0.5	0	%
0x102	Battery Voltage	0	16	0.01	0	V
0x103	Ambient Temperature	0	8	1	-40	°C

The signals were grouped into four CAN messages:

- **0x100 – VehicleStatus1:** Vehicle Speed and Engine RPM
- **0x101 – VehicleStatus2:** Coolant Temperature and Fuel Level
- **0x102 – BatteryStatus:** Battery Voltage
- **0x103 – AmbientStatus:** Ambient Temperature

2. DBC Structure

A DBC file was created to define CAN messages, signal positions, signal lengths, data types, scaling factors, offsets, ranges, and units.

The database structure is:

0x100 → VehicleStatus1
├── VehicleSpeed
└── EngineRPM

0x101 → VehicleStatus2

└─ CoolantTemperature
└─ FuelLevel

0x102 → BatteryStatus
└─ BatteryVoltage

0x103 → AmbientStatus
└─ AmbientTemperature

The DBC acts as the interpretation layer between raw CAN bytes and meaningful engineering values.

3. Validation Process

The DBC was validated by comparing the values generated by the Python transmitter with the values decoded using `cantools` and SavvyCAN.

Raw CAN traffic was monitored using:

```
candump vcan0
```

The CAN messages **0x100**, **0x101**, **0x102**, and **0x103** were successfully received and decoded.

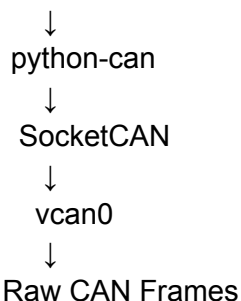
A DBC scaling experiment was also performed by changing the Battery Voltage scaling factor from **0.01** to **0.02**. The displayed engineering value changed while the raw CAN data remained unchanged. This demonstrated the importance of correct DBC scaling.

4. SocketCAN Integration

A Python transmitter using the `python-can` library was developed to generate realistic vehicle values and transmit them through the Linux virtual CAN interface `vcan0`.

The communication flow was:

Python CAN Transmitter



The transmitter generated values for:

- Vehicle Speed: **0–120 km/h**
- Engine RPM: **800–5000 rpm**

- Coolant Temperature: **20–120 °C**
- Fuel Level: **0–100 %**
- Battery Voltage: **11–15 V**
- Ambient Temperature: **-20–60 °C**

5. CAN Data Visualization

SavvyCAN was connected to the **vcan0** SocketCAN interface. The DBC file was loaded into SavvyCAN to decode the received CAN frames.

The following signals were successfully visualized:

- Vehicle Speed
- Engine RPM
- Coolant Temperature
- Fuel Level
- Battery Voltage
- Ambient Temperature

Figure 1: Raw CAN frames in SavvyCAN

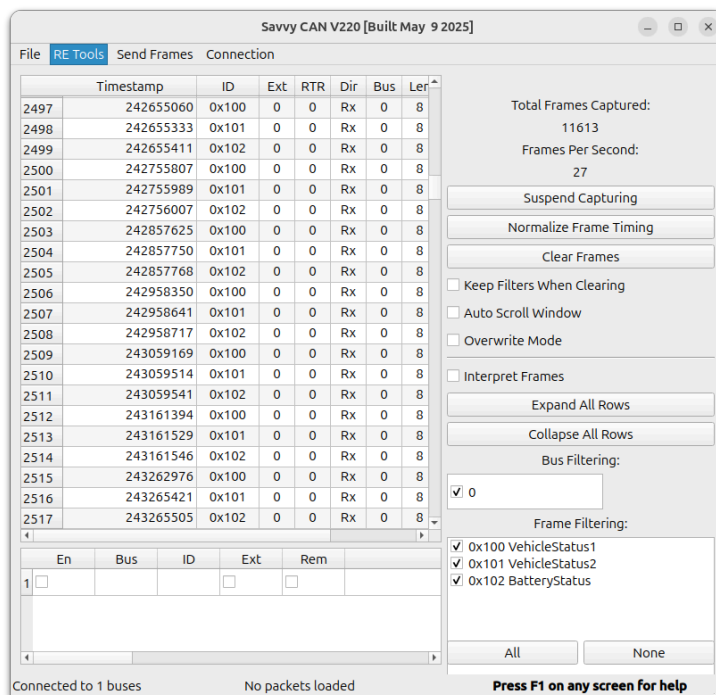


Figure 2: DBC-decoded CAN signals in SavvyCAN

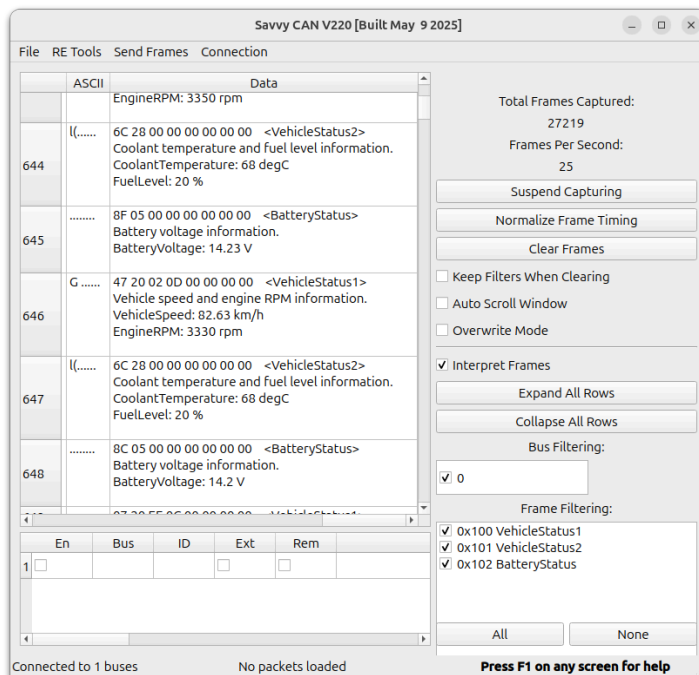
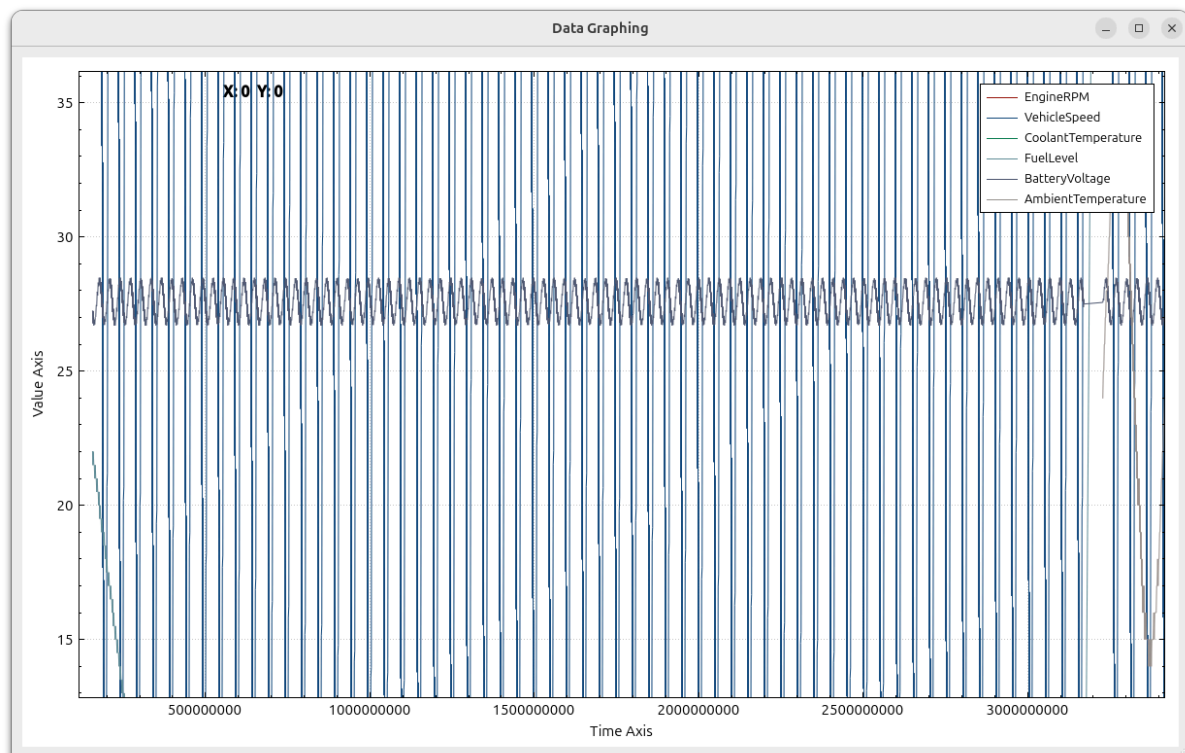


Figure 3: Live graph of the six CAN signals



6. Lessons Learned

1. SocketCAN provides the interface for sending and receiving CAN frames.
2. A DBC file defines the meaning of raw CAN data.

3. SavvyCAN can use SocketCAN and a DBC together for CAN analysis and visualization.
4. Incorrect DBC scaling or offsets can produce incorrect engineering values even when CAN communication is working correctly.
5. AI can assist with DBC generation, coding, validation, and review, but the generated content must be verified through actual CAN testing.
6. Adding a new CAN signal requires corresponding changes to the DBC, transmitter, and visualization system.

7. Conclusion

The project successfully demonstrated an end-to-end CAN communication workflow:

Signal Definition



CAN Message Design



DBC Creation



SocketCAN Transmission



Raw CAN Frames



DBC Decoding



Engineering Values



SavvyCAN Visualization

The system successfully transformed raw CAN data into meaningful automotive engineering signals and demonstrated the practical use of **DBC**, **SocketCAN**, **Python**, and **SavvyCAN** for CAN network development, testing, and analysis.